PART B — EDUCATION MODULE (replacement for Section 5.14)

> **NOTE TO AUTHORS:** This part replaces the previous Section 5.14 ("ASL Stickman Visualisations") in full. Small cross-reference edits needed elsewhere in Chapter 5 are listed at the end of this document. New bibliography entries are marked [REF: ...] where the existing reference list has no matching item.

5.14 EDUCATION MODULE: BILINGUAL SIGN-LANGUAGE LEARNING

This section documents the Education tab at the same technical depth as Part A's Translation & Chat features. The module was rebuilt during the final development cycle around motion-capture skeleton playback: signs are no longer drawn as hand-authored stick-figure keyframes but replayed from 3-D pose recordings of real signers, and the module grew from two ASL practice panels into three bilingual learning tools. The tab now offers, in both American Sign Language and Arabic Sign Language: a Sign Viewer (browse a categorised vocabulary and watch each sign, or fingerspell any typed word), a Sentence Builder (type a free-form sentence and watch it signed end-to-end), and a Practice mode (perform a prompted sign into the webcam and get instant feedback from the same recognition models that power the Chatting tab). Unlike the earlier canvas-only iteration, the Education tab is now a client–server feature: animation assembly runs in the FastAPI model-services tier, which stitches multiple sign clips into one smooth animation and streams it to the browser.

5.14.1 Overview and layout:

The Education tab is one of the two primary tabs on the App Home Page alongside Chatting (Section 5.7). A header sub-navigation carries a language toggle — "ASL · English" / "ArSL · العربية" — and a tool switch between the Sign Viewer, the Sentence Builder, and Practice mode. Switching languages remounts the tool components with a React key={lang}, so selection, playback, and camera state reset naturally without manual synchronisation logic. Each component receives its language-specific configuration (word list, categories, alphabet, text direction, placeholder copy) from a single LANG_CONFIG lookup, so the English and Arabic experiences share one code path.

[INSERT IMAGE: Education tab, Sign Viewer open in ASL mode — skeleton viewer, category tabs, word grid, Slow/Normal controls. Suggested caption: "Eshara Education tab (Sign Viewer, ASL): motion-capture skeleton playback with categorised vocabulary and playback-speed controls."]

5.14.2 Motivation: from hand-authored keyframes to motion capture:

The previous iteration of this module (documented in earlier drafts of this chapter) rendered signs with a bespoke Canvas2D stick figure: every sign was a manually authored sequence of wrist keyframes and procedurally constructed handshapes, resolved through a two-joint inverse-kinematics solver. That engine was fast and dependency-free, but it could not scale in either quantity or quality. Each new sign required careful manual authoring and a geometric verification pass (hands merging, contact points

floating off the head), the vocabulary plateaued at fifteen words, and even a correct keyframe animation reads as a symbolic illustration rather than a human performing the sign — a material pedagogical weakness for a learning tool.

The module was therefore rebuilt on skeleton recordings of real signers. Every sign is a .pose file — the standard binary format of the pose-format library [REF: pose-format] — containing per-frame MediaPipe Holistic landmarks (body, face, and both hands) extracted from video of a human signer, replayed in the browser by the <pose-viewer> web component from the sign-language-processing project (the same renderer used by the open-source sign.mt translation platform [REF: sign.mt]). This reversal of the earlier decision to avoid <pose-viewer> was deliberate and evidence-based: the original lag that motivated the Canvas2D rewrite was caused not by the renderer itself but by playing many small clips back-to-back with client-side fetches between them. The rebuilt module fixes the root cause with server-side animation stitching (Section 5.14.4) — one continuous animation per Play press — while gaining human motion data, per-finger articulation, and a content pipeline that can grow the vocabulary without manual animation work. The legacy canvas stickman remains only as a fallback renderer for the few English words that have hand-authored keyframes but no .pose file.

5.14.3 Content pipeline: dictionary signs from sign.mt:

Sign content is fetched offline from sign.mt's public dictionary API (model-services/fetch_signmt_pose.py), which translates a spoken-language word into a signed-language pose clip drawn from a curated dictionary: American Sign Language (ase) for English and Jordanian Sign Language (jos) for Arabic — the only openly available Arabic-lookup sign dictionary. The deployed vocabulary counts 44 English words plus the 26-letter manual alphabet, and 22 verified Arabic word signs plus the 28-letter Arabic alphabet. Every clip is attributed on-page (“Motion captured from a real signer”). Critically, the API silently returns a fingerspelled letter sequence, with HTTP 200, for any word its dictionary does not contain — so a successful fetch does not prove the dictionary has the sign. The verification methodology built to defeat this failure mode is documented in Section 5.14.8.

One rendering subtlety is worth recording. The pose-viewer custom element upgrades lazily, and a false JSX boolean attribute is removed from the DOM entirely — so autoplay={false} never reaches the component, whose internal default is true. Each component therefore sets el.autoplay = false imperatively in a mount effect, and drives playback through the component's play()/pause() methods and its loadeddata$ / ended$ events.

5.14.4 Server-side animation stitching (pose_concat.py):

Playing letter or word clips back-to-back on the client produced the original module's characteristic lag: between every two clips the skeleton snapped from one pose to another, with a network fetch in between. All multi-sign playback was therefore moved server-side. The FastAPI tier stitches the requested signs into ONE .pose file using spoken-to-signed — sign.mt's own concatenation package [REF: spoken-to-signed] — and streams the finished animation to the client:

1. Each clip is reduced (redundant landmarks dropped), normalised, and trimmed at its natural signing boundaries (a wrist-above-elbow heuristic finds where signing actually starts and ends).

2. Adjacent clips are cut at their best connection point — the frame pair where the two skeletons are geometrically closest — so one sign flows into the next.

3. A short zero-confidence gap (0.2 s) is inserted at every boundary and filled by interpolation; the whole sequence is then smoothed with a Savitzky–Golay filter, wrist positions are corrected, and the result is size-normalised.

Three language-parameterised endpoint families expose this pipeline:

- **/api/pose-seq (mode=words):** stitches a list of named signs with natural trimmed motion — used when the Sign Viewer plays a word.
- **/api/pose-seq (mode=spell):** fingerspelling mode. Letter clips are inconsistent upstream (some are 3-frame stills, some full 25/30 fps clips), and connection-point cutting can shrink a letter to a few unreadable frames — so spell mode instead slices each letter to its central held handshape and freezes it for 0.35 s, producing a uniform, readable spelling rhythm.
- **/api/pose-text:** free-text mode for the Sentence Builder (Section 5.14.6).

Every stitching endpoint has a /meta twin returning per-segment durations on the final timeline; because the animation itself is a single opaque file, synchronized UI feedback (highlighting the word chip or letter currently playing) is driven entirely by this metadata with plain client-side timers. Results are memoised server-side (functools.lru_cache) so a repeated Play press is a cache hit, and Content-Disposition headers use RFC 5987 encoding because Arabic filenames cannot be represented in latin-1 HTTP headers. Sentence input is bounded (12 tokens, 80 stitched segments) so a pathological request cannot exhaust the service.

5.14.5 Sign Viewer:

The Sign Viewer presents the language's vocabulary as category tabs (Everyday Words, People, Feelings, ...; كلمات يومية، الناس، مشاعر وصفات ...) with one button per word. Selecting a word previews its first frame; Play streams the sign at Slow (0.5×) or Normal (1×, the default) speed, and each word is captioned with its meaning — for Arabic, an English gloss, since the app's UI language is English. A final tab, Fingerspelling, replaces the word grid with a letter reference grid (A–Z for ASL; the 28 Arabic base letters for ArSL) whose buttons play each letter's held handshape, plus a free spelling input: any typed word is filtered to spellable letters and played as one server-stitched spell-mode sequence, with the on-screen grid highlighting each letter in sync via the /meta timing data. If the configured default word is ever pruned from the vocabulary, the component falls back to the first available word rather than rendering an empty viewer.

[INSERT IMAGE: Sign Viewer in ArSL mode mid-playback of a word sign (e.g. أم), showing RTL category tabs and Arabic word buttons. Suggested caption: “Sign Viewer (ArSL): Jordanian Sign Language skeleton playback with right-to-left category and word controls.”]

[INSERT IMAGE: Fingerspelling tab mid-spelling of a typed word with the active letter highlighted in the grid. Suggested caption: “Fingerspelling: a typed word plays as one server-stitched sequence while the current letter is highlighted from timing metadata.”]

5.14.6 Sentence Builder — free-text signing:

The Sentence Builder follows sign.mt's interaction model: the user types any sentence; words found in the sign dictionary are signed, and unknown words are fingerspelled letter by letter. The server's plan builder (pose_concat.build_text_sequence) implements this as follows:

1. The text is tokenised (lowercased; Arabic diacritics stripped; digits 1–5 and Arabic-Indic ٥–١ mapped to their number words).
2. Tokens are matched greedily against the dictionary, longest phrase first (3-gram, then 2-gram, then single word) — multi-word entries such as "thank you" or "مع السلامة" exist as ONE pose file and must not be split into unknown singles and spelled.
3. Lookups are orthography-tolerant (Section 5.14.7): both the query and the stored filenames are folded before comparison.
4. Any token still unmatched is decomposed into its letters and fingerspelled inline, using the same held-peak treatment as the Fingerspelling tab.
5. The full plan is stitched into one smooth animation and returned with per-token and per-letter timing metadata.

The frontend mirrors the same greedy matching client-side to render a live chip preview as the user types: solid chips are words that will be signed from the dictionary; dashed chips marked 🔤 will be fingerspelled. This honesty matters — a learner must never mistake a spelled word for the word's actual sign. During playback the chips and, for spelled tokens, the individual letters light up in sync with the animation.

[INSERT IMAGE: Sentence Builder with a typed Arabic sentence showing a mix of solid (signed) and dashed 🔤 (fingerspelled) chips during playback. Suggested caption: "Sentence Builder: dictionary words render as solid chips and are signed; unknown words render as dashed chips and are fingerspelled, with live highlighting driven by timing metadata."]

5.14.7 Arabic support (ArSL):

Every pose endpoint takes lang=en|ar and resolves content from a per-language directory. The tools are fully parameterised for Arabic: right-to-left text direction on inputs and labels, Arabic-Indic digit handling, and Arabic category labels. Two Arabic-specific engineering problems are worth recording:

- **Orthographic variance.** Arabic words are commonly typed without hamza/madda marks (ام for أم, اسف for آسف), yet the sign dictionary keys many entries under proper orthography — and vice versa. A single folding table (أ إ آ ٱ → ا; ى ئ → ي; ؤ → و; ة → ه; ء dropped) is applied to BOTH sides of every lookup: the server resolves queries through a folded-name index, and the client mirrors the same fold for chip previews and practice-answer matching. Files keep their proper spellings; users may type either form.

- **Fingerspelling alphabet.** ArSL fingerspelling uses the 28 base letters; variant forms fold onto the letters that have clips (e.g. ة spells as ه), and diacritics are stripped before spelling.

Two linguistic caveats are documented for the record. First, the Arabic sign content is Jordanian Sign Language (jos) — the only openly available Arabic sign dictionary — while the ArSL recognition models were trained on KArSL (Saudi) data; regional sign variants differ, so the viewer and the recogniser are not guaranteed to agree on every word. Second, sentence signing follows the typed word order (a transliteration); it does not reorder into natural sign-language grammar. Both are acceptable for a learning aid but are stated honestly in the interface and in this thesis.

5.14.8 Content verification — defeating the silent fallback:

The hardest data-quality problem in the module was the dictionary API's silent fallback: for an out-of-vocabulary word it returns a fingerspelled letter sequence that is indistinguishable, at the protocol level, from a real sign. Publishing such a clip as a word sign would teach learners something actively wrong, and visual review alone proved unreliable — a reviewer who does not know Jordanian Sign Language cannot always tell an unfamiliar sign from a spelled sequence. An automated classifier was therefore built:

1. For a candidate word, fetch the word itself AND the doubled word (the word concatenated with itself). The doubled form is guaranteed to be out-of-vocabulary, so its returned animation is, by construction, the cloud's own spelling of the word, processed by the identical pipeline.

2. Reduce both animations to per-frame handshape descriptors: the 21 right-hand landmarks, wrist-relative and scale-normalised, so global position and size differences cancel.

3. Compare with start-anchored dynamic time warping (DTW), which absorbs the timing differences the stitching pipeline introduces. If the word's animation matches the spelling's prefix, it IS the spelling fallback. Scores separate cleanly: spelled < 0.9, real ≥ 2.9.

Every accepted clip was additionally reviewed frame-by-frame as a rendered filmstrip. A sweep of roughly 170 candidate Arabic words produced two further discoveries. First, the dictionary keys many entries under proper orthography only — أم is a real sign while ام comes back spelled — which directly motivated the fold-everywhere lookup of Section 5.14.7. Second, words containing ة crash the API with HTTP 500 when out-of-vocabulary (its spelling lexicon lacks that letter), which turns the status code into a truth oracle for that word class. The final Arabic vocabulary — verified word signs only — is exactly the set that passed all checks; everything else fingerspells honestly with the dashed 🔤 treatment.

[INSERT IMAGE: Optional: a filmstrip sheet comparing a genuine word sign against its doubled-word (spelled) control. Suggested caption: “Verification filmstrips: a genuine sign (top) shows distinct arm motion; the doubled-word control (bottom) shows the fingerspelling fallback the classifier detects.”]

5.14.9 Practice mode — learn by doing:

Practice mode closes the learning loop: after watching a sign, the user performs it. The panel reuses the exact camera-and-predict pipeline that powers the live Chatting tab — extracted into a shared useCamera hook — with its own camera instance so the two tabs never interfere:

- **Letter practice:** the panel prompts a random letter; the user poses it and a single webcam frame is sent to POST /predict (mode=letters). The models return exactly the comparison target — an uppercase English letter or a real Arabic glyph — so matching is a normalised string comparison.
- **Word practice:** the panel prompts a word, runs a 3-2-1 countdown (cued performance needs reaction time), records a 3.2-second clip, and sends it to POST /predict (mode=words). Arabic matching folds orthography on both sides so the model's bare-spelled labels (لا) match the vocabulary's proper spellings (لَا).
- **Honest target lists:** only words the deployed checkpoints can actually recognise are offered for checking — the education words inside the WLASL-100 class list for ASL, and the fold-normalised intersection with the KArSL model's labels for ArSL. Everything else is watch-only rather than silently impossible.
- **Constructive feedback:** when the model saw a sign but stayed below its confidence threshold, the UI reports its best guess ("it looked like X, just not clearly enough") instead of a generic failure, distinguishing "almost had it" from "wrong sign entirely".

[INSERT IMAGE: Practice mode with the webcam active, a prompted target sign, and a correct/incorrect feedback banner. Suggested caption: "Practice mode: the learner performs a prompted sign; the same recognition models that power live translation check the attempt and give constructive feedback."]

5.14.10 Performance and verification engineering:

Four decisions keep the rebuilt module responsive and correct, summarised against the superseded Canvas2D engine in Table 5-3 (replacement below):

- **One stitched file per playback.** All multi-sign animation is assembled server-side and cached, so the client performs exactly one media fetch per Play press regardless of sentence length — the root-cause fix for the original choppy, per-clip playback.
- **A single persistent viewer element.** Each tool keeps one <pose-viewer> mounted across word and tab switches (hidden when the canvas fallback is active), so switching words never re-initialises the rendering component; idle previews load only the first frame.
- **Guarded interaction states.** Word buttons, letter grids, and inputs are disabled during playback; playback promises resolve on the component's ended$ event, a cancellation poll, or a hard timeout, so the UI can never wedge in a playing state.
- **End-to-end browser verification.** Every change to the module was verified in a scripted headless Chrome session (Playwright): log in as a QA user, drive the real UI — switch language, play specific words, type sentences, assert chip classifications — and require zero browser console errors. Sign-

content changes were additionally verified by rendering filmstrip sheets of the actual pose data and reviewing them visually.

Replacement for Table 5-3 (Education tab renderer comparison; the old table compared <pose-viewer> unfavourably and is now inverted):

- **Rendering technology:** superseded — native Canvas2D stick figure (hand-authored keyframes + IK). Deployed — <pose-viewer> skeleton playback of motion-capture .pose files.
- **Animation source:** superseded — in-repo keyframes authored by the team (15 words). Deployed — dictionary recordings of real signers via sign.mt (44 EN + 22 AR words, both alphabets), verified per clip.
- **Multi-sign playback:** superseded — per-letter client sequencing. Deployed — one server-stitched animation with timing metadata for UI highlights.
- **Bilingual coverage:** superseded — ASL only. Deployed — full ASL/ArSL parity including RTL and orthography-folded lookup.
- **Correctness assurance:** superseded — manual geometric review of keyframes. Deployed — DTW-based fallback detection plus filmstrip review plus scripted zero-console-error browser tests.

Part B summary (replacement):

The Education module pairs three tools — a categorised Sign Viewer with integrated fingerspelling, a free-text Sentence Builder, and a webcam Practice mode — across two sign languages behind one bilingual architecture. Signs are motion-capture skeletons of real signers (ASL from sign.mt's ase dictionary; Jordanian Sign Language for Arabic), rendered by the pose-viewer web component and stitched server-side into single smooth animations whose timing metadata drives synchronised UI highlights. Arabic support required orthography-folded lookup on both client and server, right-to-left interface treatment, and a rigorous DTW-based verification methodology to reject the dictionary API's silent fingerspelling fallbacks — ensuring that every word the module presents as a sign is genuinely that sign, and that everything else is honestly labelled as fingerspelling. Practice mode closes the loop by checking the learner's own signing against the same recognition models that power live translation, offering only targets those models can actually recognise.

Cross-reference edits elsewhere in Chapter 5

NOTE TO AUTHORS: Apply these small edits so Part A stays consistent with the new Part B:

1. Section 5.2.3 (Stickman visualisation interface): replace the single bullet with: “Sign playback and practice (pose-viewer + webcam) — the Educational tab renders motion-capture skeleton playback through the <pose-viewer> web component, assembled server-side by the model-services tier, alongside a webcam practice panel; the architecture is documented in full in Section 5.14 (Part B).” Retitle the subsection “Education module interface”.

2. Section 5.1 / Figure 5-1 and Section 5.5: no structural change — but note that the FastAPI tier now also serves the /api/poses and /api/pose-seq · /api/pose-text stitching endpoints consumed by the Education tab (already hinted in Section 5.1's Inference Tier description).

3. Section 5.8 (Educational tools paragraph): replace with: “Educational tools. The Educational tab hosts three bilingual learning tools — a Sign Viewer with fingerspelling, a free-text Sentence Builder, and a webcam Practice mode — built on motion-capture skeleton playback with server-side animation stitching; see Section 5.14 (Part B).”

4. Figure 5-6 (use case diagram): update the Educational module bubble labels to “Browse & watch word signs”, “Fingerspell typed text”, “Sign a typed sentence”, and “Practice signs with webcam feedback”.

5. Figure 5-10: replace the old two-panel screenshot with the new Education tab screenshots called out in Section 5.14 above.

6. Chapter 5 summary paragraph: replace the Part B sentence with: “Part B documented the Education Module: bilingual Sign Viewer, Sentence Builder, and Practice tools built on motion-capture skeleton playback (pose-viewer), server-side animation stitching with timing metadata, orthography-folded Arabic lookup, and a DTW-based content verification methodology that rejects fingerspelling fallbacks.”

7. Bibliography: add entries for [REF: sign.mt] (Moryossef & Müller, sign.mt — open-source multilingual sign translation platform, https://sign.mt), [REF: pose-format] (sign-language-processing pose-format library, https://github.com/sign-language-processing/pose), and [REF: spoken-to-signed] (spoken-to-signed-translation pipeline, https://github.com/sign-language-processing/spoken-to-signed-translation). The existing MediaPipe references [4, 5] still apply to the .pose landmark data.

8. Section 5.14 in the Table of Contents / List of Figures: update subsection titles to match the new 5.14.1–5.14.10 headings and re-caption Figures 5-10–5-13 (the IK and keyframe-timeline figures 5-11–5-13 describe the superseded engine and should be removed or moved to the superseded-experiments discussion in Chapter 7).